

观看文艺汇演问题

题目描述

为了庆祝中国共产党成立 100 周年，某公园将举行多场文艺表演，很多演出都是同时进行。

一个人只能同时观看一场演出，且不能迟到早退。

由于演出分布在 **不同的** 演出场地，所以连续观看的演出最少有 15 分钟的时间间隔。

小明是一个狂热的文艺迷，想观看尽可能多的演出。

现给出演出时间表，请帮小明计算他最多能观看几场演出。

输入描述

第一行为一个数 N ，表示演出场数

- $1 \leq N \leq 1000$

接下来 N 行，每行有被空格分割的整数，第一个整数 T 表示演出的开始时间，第二个整数 L 表示演出的持续时间

- T 和 L 的单位为分钟
- $0 \leq T \leq 1440$
- $0 < L \leq 180$

输出描述

输出最多能观看的演出场数

用例

输入	2 720 120 840 120
输出	1
说明	两场演出间隔时间为0，不满足最小15分钟时间间隔的要求，所以最多只能观看一场演出

输入	2 0 60 90 60
输出	2
说明	两场演出间隔大于15分钟，都能观看到

题目解析

本题其实是区间问题。且是要求解**最多不相交区间个数**。

需要注意的是，两个区间不相交的条件也比较特殊：

连续观看的演出最少有15分钟的时间间隔

即两个区间之间需要间隔15单位长度才算不相交。

JS算法源码

```
1 | const rl = require("readline").createInterface({ input: process.stdin });
2 | var iter = rl[Symbol.asyncIterator]();
3 | const readline = async () => (await iter.next()).value;
```

运行

```

4 | 5 | void (async function () {
6 |   const N = parseInt(await readline());
7 |
8 |   const times = [];
9 |   for (let i = 0; i < N; i++) {
10 |     const [T, L] = (await readline()).split(" ").map(Number);
11 |     times.push([T, T + L]);
12 |   }
13 |
14 |   // 按照演出结束时间升序
15 |   times.sort((a, b) => a[1] - b[1]);
16 |
17 |   // 最多能看的演出场数，第一场演出必可以看
18 |   let count = 1;
19 |
20 |   let last_end = times[0][1];
21 |   for (let i = 1; i < times.length; i++) {
22 |     const [start, end] = times[i];
23 |
24 |     // 如果当前演出开始时间 和 上一次演出结束时间，相隔不低于15分钟，则可以看当前演出
25 |     if (start - last_end >= 15) {
26 |       count++;
27 |       last_end = end;
28 |     }
29 |   }
30 |
31 |   console.log(count);
32 | })();

```

Java算法源码

```

1 | import java.util.Arrays;
2 | import java.util.Scanner;

```

```
3 | 4 public class Main {
4 |     public static void main(String[] args) {
5 |         Scanner sc = new Scanner(System.in);
6 |
7 |         int n = sc.nextInt();
8 |
9 |
10 |         int[][] times = new int[n][2];
11 |         for (int i = 0; i < n; i++) {
12 |             int T = sc.nextInt();
13 |             int L = sc.nextInt();
14 |
15 |             times[i][0] = T; // 开始时间
16 |             times[i][1] = T + L; // 结束时间 = 开始时间 + 持续时间
17 |         }
18 |
19 |         // 按照演出结束时间升序
20 |         Arrays.sort(times, (a, b) -> a[1] - b[1]);
21 |
22 |         // 最多能看的演出场数，第一场演出必可以看
23 |         int count = 1;
24 |
25 |         int last_end = times[0][1];
26 |         for (int i = 1; i < times.length; i++) {
27 |             int start = times[i][0];
28 |             int end = times[i][1];
29 |
30 |             // 如果当前演出开始时间 和 上一次演出结束时间，相隔不低于15分钟，则可以看当前演出
31 |             if (start - last_end >= 15) {
32 |                 count++;
33 |                 last_end = end;
34 |             }
35 |         }
36 |
37 |         System.out.println(count);
38 |     }
39 | }
```

Python算法源码

```
1  # 输入获取
2  N = int(input())
3
4  times = []
5  for _ in range(N):
6      T, L = map(int, input().split())
7      times.append([T, T + L])
8
9
10 # 算法入口
11 def solution():
12     # 按照演出结束时间升序
13     times.sort(key=lambda time: time[1])
14
15     # 最多能看的演出场数，第一场演出必可以看
16     count = 1
17
18     last_end = times[0][1]
19     for i in range(1, N):
20         start, end = times[i]
21
22         # 如果当前演出开始时间 和 上一次演出结束时间，相隔不低于15分钟，则可以看当前演出
23         if start - last_end >= 15:
24             count += 1
25             last_end = end
26
27     return count
28
29
30 # 输出打印
31 print(solution())
```

C算法源码

运行

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int cmp(const void *a, const void *b) {
5      return ((int *) a)[1] - ((int *) b)[1];
6  }
7
8  int main() {
9      int N;
10     scanf("%d", &N);
11
12     int times[N][2];
13     for (int i = 0; i < N; i++) {
14         int T, L;
15         scanf("%d %d", &T, &L);
16
17         times[i][0] = T; // 开始时间
18         times[i][1] = T + L; // 结束时间 = 开始时间 + 持续时间
19     }
20
21     // 按照演出结束时间升序
22     qsort(times, N, sizeof(times[0]), cmp);
23
24     // 最多能看的演出场数，第一场演出必可以看
25     int count = 1;
26
27     int last_end = times[0][1];
28     for (int i = 1; i < N; i++) {
29         int start = times[i][0];
30         int end = times[i][1];
31
32         // 如果当前演出开始时间 和 上一次演出结束时间，相隔不低于15分钟，则可以看当前演出
33         if (start - last_end >= 15) {
34             count++;
35             last_end = end;
```

```

36 |         }
    |         37 |     }
38 |
39 |     printf("%d\n", count);
40 |
41 |     return 0;
42 | }

```

C++算法源码

```

1 | #include <bits/stdc++.h>
2 |
3 | using namespace std;
4 |
5 | int main() {
6 |     int N;
7 |     cin >> N;
8 |
9 |     vector<vector<int>> times;
10 |
11 |     for (int i = 0; i < N; i++) {
12 |         int T, L;
13 |         cin >> T >> L;
14 |
15 |         times.emplace_back(vector<int>{T, T + L});
16 |     }
17 |
18 |     // 按照演出结束时间升序
19 |     sort(times.begin(), times.end(), [](vector<int> &a, vector<int> &b) {
20 |         return a[1] < b[1];
21 |     });
22 |
23 |     // 最多能看的演出场数，第一场演出必可以看
24 |     int count = 1;

```

```
25 |
26 |     int last_end = times[0][1];
27 |     for (int i = 1; i < N; i++) {
28 |         int start = times[i][0];
29 |         int end = times[i][1];
30 |
31 |         // 如果当前演出开始时间 和 上一次演出结束时间，相隔不低于15分钟，则可以看当前演出
32 |         if (start - last_end >= 15) {
33 |             count++;
34 |             last_end = end;
35 |         }
36 |     }
37 |
38 |     cout << count << endl;
39 |
40 |     return 0;
41 | }
```